

Software Requirements Specification (SRS)

1. Introduction

1.1 Purpose

The purpose of this document is to define the functional, non-functional, and system requirements for the Railway Reservation System (RRS). This system is intended to provide passengers with an online platform for booking train tickets, checking train schedules, and managing reservations efficiently.

1.2 Scope

The Railway Reservation System is designed to automate the ticket booking process for railway services. It allows users to:

- Search for train schedules.
- Book, cancel, and modify tickets.
- View seat availability and fare details.
- Receive confirmation via email or SMS.

The system is intended for passengers, railway administrators, and customer service representatives. It supports web and mobile platforms.

1.3 Definitions, Acronyms, and Abbreviations

- **RRS:** Railway Reservation System
- **PNR:** Passenger Name Record
- **UI:** User Interface
- **Admin:** System Administrator
- **IEEE:** Institute of Electrical and Electronics Engineers

1.4 References

- IEEE Std 1830-1998: Recommended Practice for Software Requirements Specifications.
- Railway system operational guidelines.

1.5 Overview

This document is structured to provide detailed system functionalities, design constraints, assumptions, and dependencies to guide the development process.

2. Overall Description

2.1 Product Perspective

The Railway Reservation System will replace the manual ticketing process, integrating with railway databases for real-time updates. It will be accessible via web browsers and mobile applications.

2.2 Product Features

- **Train Search:** Search by train number, name, or route.
- **Booking and Cancellation:** Book tickets or cancel reservations.
- **Payment Gateway:** Multiple options like credit card, UPI, and net banking.
- **PNR Status:** Track booking status.
- **Admin Module:** Manage train schedules and user complaints.

2.3 User Characteristics

- **Passengers:** General public with basic computer or smart phone literacy.
- **Admin Users:** Railway staff with system training.
- **Customer Support:** Handle passenger complaints and assist with bookings.

2.4 Constraints

- System should process 1000+ simultaneous transactions.
- Real-time synchronization with railway databases.
- Must comply with data protection regulations.

2.5 Assumptions and Dependencies

- Internet connectivity is required for all user operations.
 - Railway schedules are maintained externally and integrated through APIs.
 - Users must register to book tickets.
-

3. Specific Requirements

3.1 Functional Requirements

1.

User Registration and Login

2.

- Users can register with personal details.
- Login is authenticated using email/phone and password.

3.

Train Search and Seat Availability

4.

- Search trains by source, destination, and date.
- Display seat availability, fare, and class details.

5.

Ticket Booking and Cancellation

6.

- Book tickets by selecting train, class, and passenger details.
- Generate a unique PNR for every booking.
- Allow cancellations with refund calculations based on policies.

7.

PNR Status Check

- Users can view current booking status (confirmed, waitlisted, or canceled).

Payment Integration

- Secure payment options like debit/credit card, UPI, and wallet services.
- Provide payment receipts post-confirmation.

Admin Module

- Add/update train schedules.
- Manage seat availability and ticket policies.

3.2 Non-Functional Requirements

1. Performance

- System should handle 5000 concurrent users.

2. Reliability

- 99.9% uptime.

3. Security

- Use HTTPS for secure communication.
- Encrypt sensitive data like passwords and payment details.

4. Usability

- User-friendly interface for web and mobile platforms.

5. Scalability

- Ability to add more trains/routes without downtime.

3.3 Interface Requirements

- **User Interface:** Responsive design for both mobile and desktop users.
- **API Interface:** Connects with railway databases for real-time updates.

3.4 System Constraints

- Limited by railway database updates.
- Adherence to railway policies for refunds and cancellations.

4. Appendices

- Use cases for ticket booking and cancellation.
 - Sample UI mockups for the booking interface.
 - System architecture diagram for reference.
-

This document can be expanded further based on specific requirements or customization needs. Let me know if you need additional sections!

SDLC Model (Software Development Life Cycle)

SDLC ek process hai jo software development ke har stage ko define karta hai, jisme **planning, designing, implementation, testing, deployment, aur maintenance** included hote hain. Ye ek systematic approach hai jo software development ko organize aur manage karne ke liye use hota hai. Popular SDLC models mein Waterfall, Agile, Spiral, aur V-Model शामिल hain.

Example: Waterfall Model ek linear approach hai jisme har phase sequentially execute hota hai, jabki Agile ek iterative model hai jo frequent updates aur feedback par focus karta hai.

Advantages of SDLC:

1. **Clear Structure:** Har phase ke liye well-defined deliverables hote hain.
2. **Risk Management:** Spiral model jaisa SDLC risk ko manage karne mein madad karta hai.
3. **Efficient Resource Use:** Development resources ka efficient allocation hota hai.
4. **Quality Control:** Testing aur review har phase mein ensure hota hai.
5. **Improved Documentation:** Documentation se maintenance aur future updates easy hote hain.

Disadvantages of SDLC:

1. **Time-Consuming:** Har phase ka completion sequentially hone ke karan zyada time lagta hai.
2. **Costly:** Changes implement karna expensive ho sakta hai.
3. **Rigid:** Waterfall jaise models flexibility nahi dete.
4. **Complexity:** Spiral aur Agile jaise models complex ho sakte hain.
5. **Dependency on Initial Requirements:** Agar requirements galat ho, toh pura system fail ho sakta hai.

Software Quality Assurance (SQA)

SQA ek process hai jo ensure karta hai ki software products high-quality standards ko meet karein. Is process mein planning, auditing, testing, aur monitoring शामिल hain taaki bugs aur defects ko minimize kiya ja sake. Ye SDLC ke har phase mein apply hota hai aur software reliability, performance, aur security ko ensure karta hai.

Example: Testing teams ka use karna jo functionality aur performance tests perform karte hain, aur automation tools jaise Selenium aur JIRA ko use karte hain.

Use Case Diagram

Use Case Diagram ek graphical representation hai jo system ke functional requirements ko define karta hai. Is diagram mein actors (users ya external systems) aur unke actions ko show kiya jata hai. Ye UML (Unified Modeling Language) ka ek part hai aur system ki functionality ko understand karne mein madad karta hai.

Types of Use Case Diagrams:

1. **System Use Case Diagram:** High-level system functionality ko show karta hai.
2. **Business Use Case Diagram:** Organization processes aur workflows ko define karta hai.

Example: A **Library Management System** ka use case diagram show karega ki ek user books search kar sakta hai, borrow kar sakta hai, aur due fees check kar sakta hai.

Advantages of Use Case Diagrams:

1. **Clarity:** System requirements ko visually explain karta hai.
2. **Communication Tool:** Developers aur stakeholders ke beech understanding improve karta hai.
3. **Modularization:** System components ko logically divide karna easy hota hai.
4. **Requirement Validation:** Functional requirements ko verify karna easy hota hai.
5. **Standardized Representation:** UML standards follow karne se global compatibility hoti hai.

Disadvantages of Use Case Diagrams:

1. **Limited Scope:** Non-functional requirements ko represent nahi karta.
2. **Complex Systems:** Large systems ke liye diagram cluttered ho sakta hai.

3. **No Sequence Details:** Actions ke sequence ko nahi dikhata.
4. **Dependency on Skills:** Correct representation ke liye skilled designers chahiye.
5. **Updates Required:** Frequent changes ke wajah se diagram outdated ho sakta hai.

Software (Definition, Advantages, Disadvantages, and Metrics):

Software ek set of instructions (programs) hai jo ek computer system ko specific tasks perform karne ke liye guide karta hai. Ye system software (e.g., operating systems) aur application software (e.g., word processors) ke roop mein hota hai. Iska primary goal ek particular problem ko solve karna ya user ki zarurat ko fulfill karna hai.

Advantages of Software:

1. Tasks automate karne aur human efforts ko reduce karne mein madad karta hai.
2. Efficiency aur productivity increase karta hai.
3. Information ko securely store aur retrieve kar sakte hain.
4. Large-scale data processing ko enable karta hai.
5. User-friendly interfaces ke through complex problems ko simplify karta hai.

Disadvantages of Software:

1. Maintenance aur updates ke liye cost aur effort lagta hai.
2. Bugs aur vulnerabilities security risks badhate hain.
3. Compatibility issues naye hardware ya software ke saath ho sakte hain.
4. Over-dependence productivity impact kar sakta hai jab software fail ho jaye.
5. Piracy aur unauthorized copying common challenges hain.

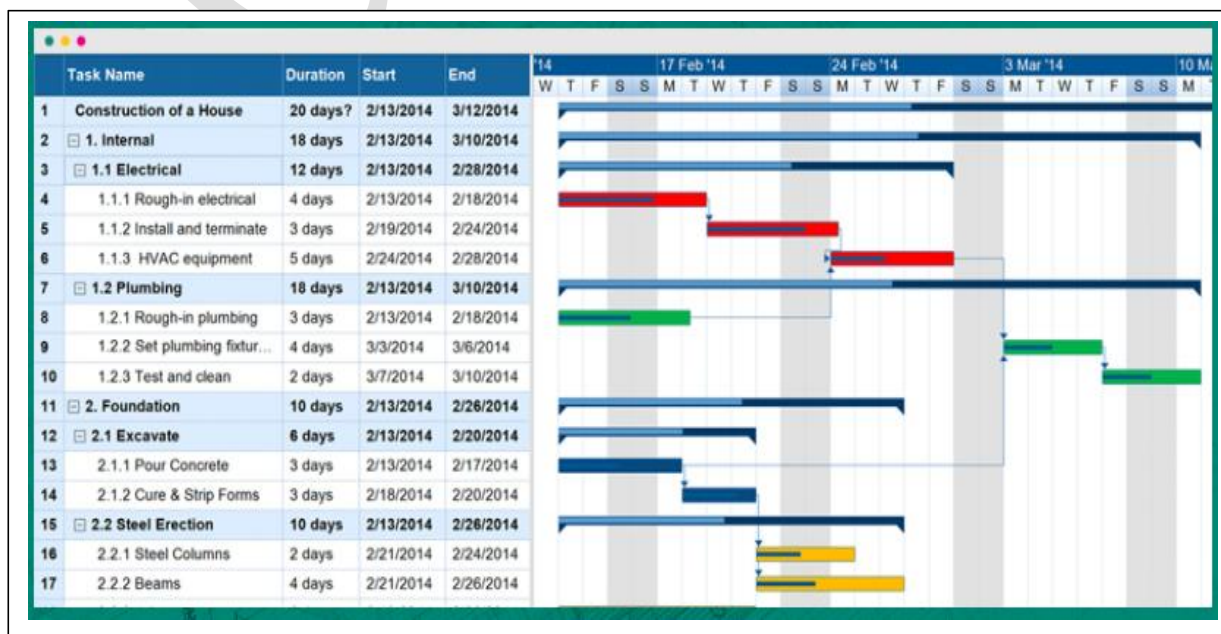
Metrics for Software:

1. **Reliability:** Failure-free operation ka measure. Example: Crash rate.
2. **Maintainability:** System ko repair ya update karne ki ease. Example: Code readability.
3. **Performance:** Response time aur throughput. Example: Load testing results.
4. **Usability:** User satisfaction ka level. Example: User surveys.
5. **Scalability:** System additional users ya workload handle karne ki ability. Example: Number of concurrent users.

Example: A web-based application like Google Drive jisme users apni files ko upload, share, aur collaborate karte hain.

Gantt Chart (Definition, Advantages, Disadvantages, and Metrics):

Gantt Chart ek bar graph representation hai jo project ke schedule aur progress ko dikhata hai. Isme horizontal bars tasks ko aur unka duration ko represent karte hain. Iska use project management mein hota hai.



Advantages of Gantt Chart:

1. Projects ka visualization aur planning easy banata hai.
2. Task dependencies aur deadlines ko clearly dikhata hai.
3. Resource allocation ko optimize karta hai.
4. Team communication aur collaboration ko enhance karta hai.
5. Progress tracking aur reporting ke liye helpful hai.

Disadvantages of Gantt Chart:

1. Complex projects ke liye cluttered aur confusing ho sakta hai.
2. Regular updates karna time-consuming hota hai.
3. Dependencies ya delays track karna difficult ho sakta hai.
4. Team members ke individual efforts ka representation nahi deta.
5. Advanced tools expensive ho sakte hain.

Metrics for Gantt Chart:

1. **On-time Completion:** Task deadlines ka adherence.
2. **Task Dependency:** Number of interdependent tasks.
3. **Resource Utilization:** Resources ki efficiency.
4. **Task Duration Variance:** Actual vs. estimated time comparison.
5. **Critical Path Analysis:** Project ka longest duration path.

Example: Software development lifecycle ke phases jaise planning, development, testing, aur deployment ka visualization.

PERT Chart Kya Hai?

PERT (Program Evaluation and Review Technique) ek project management tool hai jo project ke tasks aur unke completion time ko analyze aur schedule karne ke liye use hota hai. Ye complex projects ke planning aur execution ko simplify karta hai by visualizing the workflow aur dependencies.

Key Points:

1. Tasks ko nodes (circles/rectangles) aur dependencies ko arrows se dikhaya jata hai.
2. Time estimates: Optimistic, Most Likely, aur Pessimistic.
3. Critical Path: Sabse zyada time lagne wale tasks ka sequence jo project completion ko delay kar sakta hai.

PERT Chart Ke Steps:

1. **Tasks Define Karo:** Sabhi activities ko identify karo jo project ke completion ke liye zaroori hain.
 2. **Sequence Identify Karo:** Decide karo ki kaunsa task pehle complete hoga aur kaunsa us par dependent hai.
 3. **Dependencies Chart Karo:** Tasks aur dependencies ko visual form me draw karo using nodes aur arrows.
 4. **Time Estimate Karo:**
 1. **Optimistic Time (O):** Minimum time agar sab kuch perfect ho.
 2. **Most Likely Time (M):** Normal time jo usually lagta hai.
 3. **Pessimistic Time (P):** Maximum time agar delays ho.
 4. Formula for Expected Time: $TE = \frac{O + 4M + P}{6}$ $TE = \frac{O + 4M + P}{6}$
 5. **Critical Path Calculate Karo:** Find karo wo tasks ka path jisme sabse zyada time lagta hai. Ye path project ko delay kar sakta hai.
-

Example:

Hospital Management System ka PERT Chart

Tasks:

1. Requirements Gathering
2. System Design
3. Database Setup
4. Frontend Development
5. Backend Development
6. Testing
7. Deployment

Dependencies:

- System Design tabhi shuru hoga jab Requirements Gathering complete hoga.
- Frontend aur Backend Development parallel chal sakte hain.
- Testing ke baad hi Deployment hoga.

Steps in Action:

1. Nodes represent karte hain tasks, e.g., Node A = Requirements Gathering.
2. Arrows show karte hain dependencies, e.g., A → B (Requirements → Design).
3. Time Estimates:
 - Requirements Gathering: O=2, M=3, P=5 (TE = 3.33 days).
 - Design: O=4, M=5, P=7 (TE = 5.17 days).

Critical Path Analysis:

Longest path = Requirements → Design → Testing → Deployment.

PERT Chart Ka Process:

1. PERT ka use planning aur scheduling ke liye hota hai.
2. Complex projects ko breakdown kar ke manageable tasks me convert karta hai.
3. Dependencies aur bottlenecks identify karna easy hota hai.
4. Efficient resource allocation aur timeline optimization me help karta hai.

System Testing (Definition, Advantages, Disadvantages, and Metrics):

System Testing software development ka ek phase hai jisme complete system ko end-to-end test kiya jata hai taaki ensure ho ki system functional aur non-functional requirements ko meet karta hai.

Advantages of System Testing:

1. End-to-end functionality ko verify karta hai.
2. Bugs ko production mein jaane se pehle detect karta hai.
3. Non-functional requirements jaise performance aur security ka test karta hai.
4. System reliability aur robustness ensure karta hai.
5. User experience ko improve karta hai.

Disadvantages of System Testing:

1. Time aur cost zyada lagta hai, especially for large systems.
2. Complex systems ke liye exhaustive testing possible nahi hoti.
3. Dependency hoti hai lower-level testing ke results par.
4. Automation setup ke liye significant effort lagta hai.
5. Continuous changes ko manage karna difficult ho sakta hai.

Metrics for System Testing:

1. **Test Coverage:** Covered vs. uncovered requirements.
2. **Defect Density:** Found defects per module.
3. **Execution Time:** Test case completion time.

4. **Pass/Fail Rate:** Successful vs. failed test cases.

5. **Customer Satisfaction:** Post-testing feedback.

Example: E-commerce website testing, jisme login, product search, cart, aur payment gateway ka end-to-end validation hota hai.

Cohesion and Coupling (Definition, Advantages, Disadvantages, and Metrics):

Cohesion ek module ke andar ke components ke relationship ko represent karta hai, yaani ek module ke functions kitne tightly related hain. High cohesion preferable hoti hai.

Cohesion ka matlab hai ki ek module ya class mein kitna samanjasya hai. Ek module mein cohesion ki degree is baat par nirbhar karti hai ki usmein upasthit sabhi elements kitne achhe se judhe huye hain.

Ex : agar hum ek banking system bana rahe hain, toh ek module jo sirf account balance ko update karta hai, usmein cohesion ki degree uchhi hogi.

Cohesion ki types:

- Functional cohesion (sabse uchhi)
- Sequential cohesion
- Communicational cohesion
- Procedural cohesion
- Temporal cohesion (sabse niche)

Coupling alag modules ke beech ke dependence ko dikhata hai. Low coupling preferable hoti hai, kyunki loosely coupled systems zyada modular aur maintainable hote hain.

Coupling ka matlab hai ki do ya adhik modules ya classes kitne prabhavit karte hain ek dusre par. Coupling ki degree is baat par nirbhar karti hai ki do modules mein kitni jyada interaction hoti hai.

Ex: agar hum ek e-commerce system bana rahe hain, toh agar ek module jo order ko process karta hai, aur doosra module jo payment ko handle karta hai, toh in dono modules mein coupling ki degree uchhi hogi.

Coupling ki types:

- Tight coupling (sabse uchhi)
- Loose coupling (sabse niche)
- Content coupling
- Common coupling
- External coupling

Advantages of High Cohesion and Low Coupling:

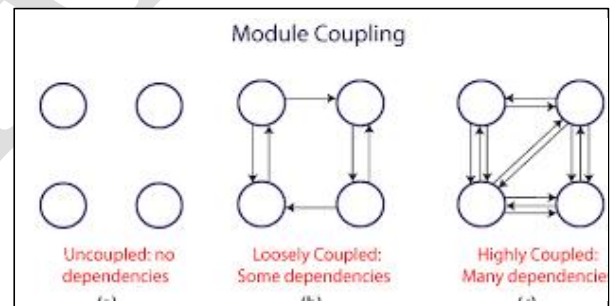
1. System zyada modular aur easy-to-maintain hota hai.
2. Debugging aur testing simple hoti hai.
3. Code reusability improve hoti hai.
4. Change management easy hota hai.
5. System ki reliability badh jati hai.

Disadvantages of High Coupling and Low Cohesion:

1. Code complexity badhti hai.
2. Dependency ki wajah se ek module ka failure dusre modules ko impact karta hai.
3. Maintenance aur scalability mushkil hoti hai.
4. Testing process me zyada time lagta hai.
5. Code readability aur understandability reduce hoti hai.

Metrics for Cohesion and Coupling:

1. **Cohesion Index:** Module functions ki relatedness measure karta hai.
2. **Coupling Degree:** Inter-module dependencies ka level.
3. **Cyclomatic Complexity:** Code complexity measure karta hai.
4. **Module Independence:** Module kitne independent hain.



5. **Code Reusability:** Repeatable components ka percentage.

Example: Library management system, jisme ek module book inventory handle karta hai aur dusra module user records. In modules ke beech low coupling aur high cohesion hona chahiye.

OPAKASH